

Facultad: Uader Fcyt

Materia: Programación avanzada

Profesor: Ernesto Ledesma

Estudiante: Romero Gonzalo Lionel

Tp: TP1 - Introducción a la Web - Arquitectura y Protocolos

Fecha de entrega: 10/08/26

1. ¿Cómo surgió el sistema de documentos distribuidos WWW?

Surgió a finales de la década de 1980 en el CERN (Organización Europea para la Investigación Nuclear). Nació para resolver la heterogeneidad y fragmentación de la información producida por científicos de todo el mundo. La propuesta buscó unificar el acceso mediante un sistema de hipertexto distribuido, permitiendo a los usuarios navegar entre documentos alojados en diferentes computadoras interconectadas mediante hipervínculos, sin importar la arquitectura de hardware o el sistema operativo subyacente.

2. ¿En qué año se creó la www? ¿Quién la creó? ¿Cuál fue el propósito inicial?

Se creó en 1989 por Tim Berners-Lee con la colaboración de Robert Cailliau buscando facilitar el intercambio automático de información y documentación científica entre investigadores e instituciones académicas conectadas al CERN.

3. ¿Sobre qué red funcionaba? ¿Cuáles fueron los componentes iniciales? WWW en la actualidad.

Funcionaba sobre internet con protocolos en TCP/IP.

Sus componentes actuales son:

HTML (*HyperText Markup Language*): Para la estructuración de documentos de hipertexto.

HTTP (*HyperText Transfer Protocol*): Protocolo de nivel de aplicación para la transferencia de archivos.

URI/URL (*Uniform Resource Identifier/Locator*): Sistema global de direccionamiento de recursos.

Servidor Web (*httpd*): Ejecutado originalmente en una estación de trabajo **NeXTcube**.

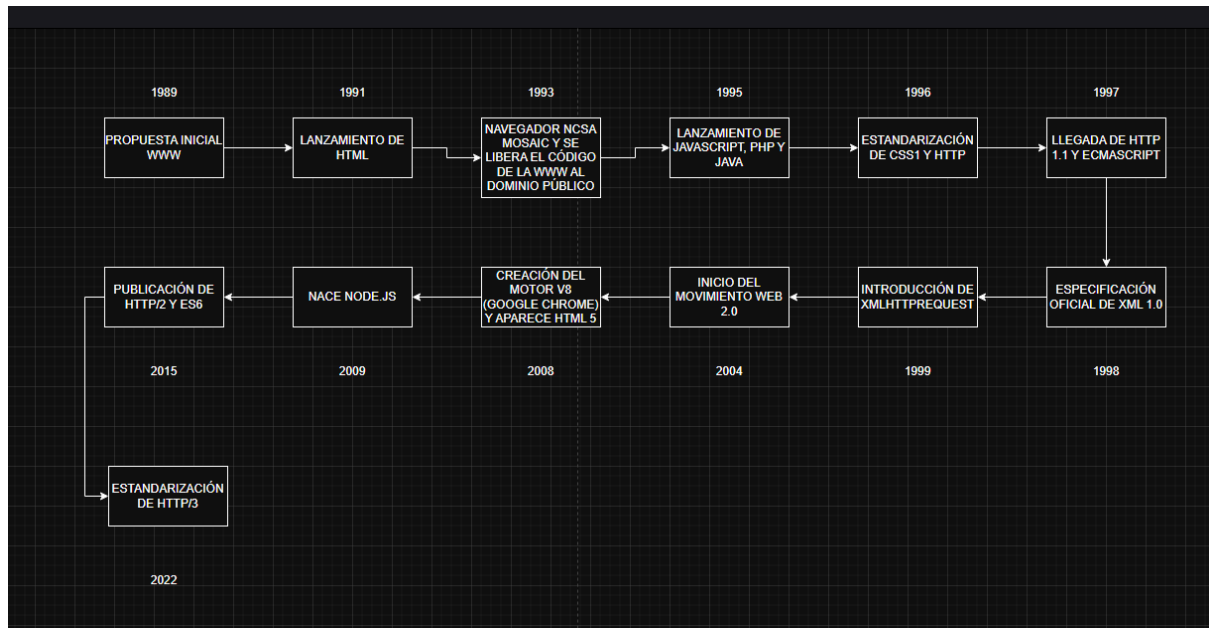
Navegador/Editor Web: Denominado *WorldWideWeb* (luego renombrado a *Nexus*).

4. Nombre y describa las distintas etapas de evolución en la web. Realice una cronología de tecnologías que surgieron en la web hasta la actualidad.

Etapas de evolución:

- **Web 1.0 (*World Wide Web Estática*):** Caracterizada por la unidireccionalidad, donde el usuario final consumía archivos HTML inalterables alojados en servidores tradicionales. La interacción era prácticamente inexistente.
- **Web 2.0 (*Web Social y Participativa*):** Marcó la transición hacia la bidireccionalidad mediante plataformas de interacción social y servicios basados en la colaboración. Tecnológicamente, se apoyó en AJAX, bases de datos dinámicas y el modelo SOA.
- **Web 3.0 (*Web Semántica*):** Enfocada en la optimización del procesamiento de datos para máquinas. Introdujo lenguajes de marcado avanzados como RDF y microdatos, facilitando la interoperabilidad del conocimiento y el uso intensivo de APIs REST.
- **Web 4.0 (*Web Ubicua e Inteligente*):** Fase actual orientada a la conectividad total y la integración de sistemas expertos. Basada en arquitecturas de microservicios, dispositivos móviles permanentes, IoT y aplicaciones web progresivas (PWA).

CRONOLOGIA:



5. ¿Qué es un Servidor WWW?

Un Servidor Web es un componente del sistema distribuido compuesto por hardware (la máquina conectada a la red) y software (proceso daemon como Apache, Nginx o IIS) que procesa peticiones entrantes bajo el protocolo HTTP/HTTPS. Su función principal es despachar recursos (archivos HTML, imágenes, JSON, scripts) hacia los clientes solicitantes, ejecutando lógica del lado del servidor (backend) si el recurso así lo requiere.

6. ¿Qué es un cliente web? Explique detalladamente.

Un Cliente Web es el proceso cliente en la arquitectura Cliente/Servidor.

Funciona de la siguiente manera:

1. **Captura la interacción:** Recibe una entrada del usuario (una URL o un clic en un hipervínculo).
2. **Resolución de nombres:** Solicita al sistema DNS la traducción del FQDN a una dirección IP.
3. **Petición del recurso:** Establece una conexión socket TCP (o QUIC en HTTP/3) con el servidor e inicia una transacción HTTP enviando cabeceras y parámetros.
4. **Renderizado:** Recibe la respuesta HTTP, interpreta el código HTML, parsea hojas de estilo (CSS), ejecuta código scripts (JavaScript) en su motor interno y renderiza el Document Object Model (DOM) resultante en pantalla.

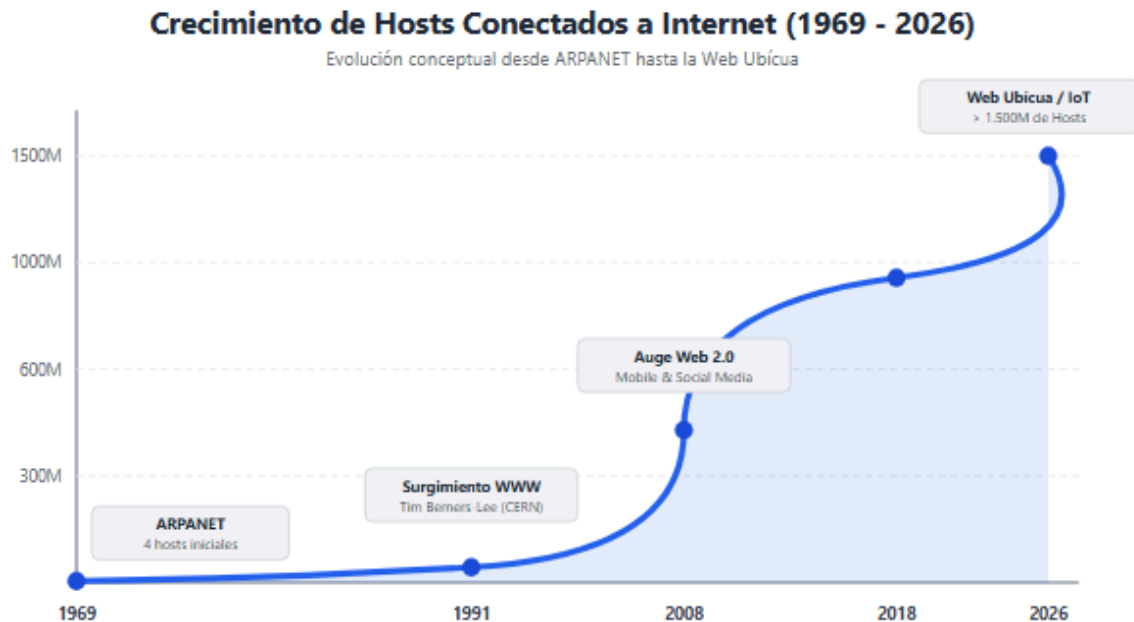
7. ¿Cómo está compuesto el sistema WWW actual?

El sistema WWW actual opera bajo un modelo distribuido multicapa (*N-tier*) integrado por:

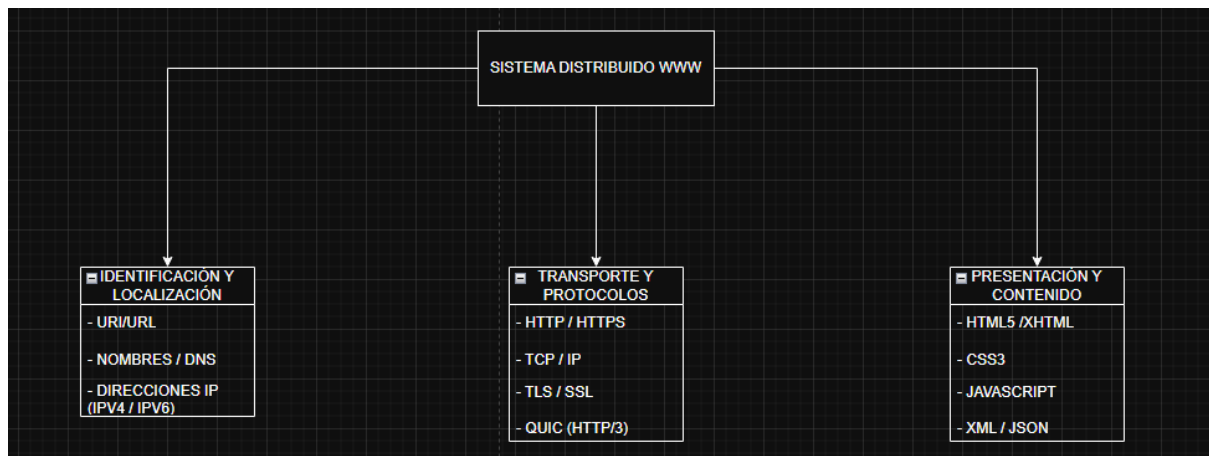
- **Clientes Web (User-Agents):** Navegadores, aplicaciones móviles y bots.
- **Infraestructura de Red y DNS:** Enrutadores, CDN (*Content Delivery Networks*) y resolvers de dominio.
- **Servidores Web / Reverse Proxies:** Nginx, HAProxy, Cloudflare (para balanceo de carga y seguridad).
- **Servidores de Aplicación (Backend):** Entornos de ejecución como Node.js, JVM, .NET, Python/Django.

- **Servicios de Almacenamiento:** Bases de datos relacionales (PostgreSQL, MySQL) y NoSQL (MongoDB, Redis).
- **Servicios y APIs Web:** Servicios REST, GraphQL y WebSockets.

8. Realice un diagrama de crecimiento de host conectados a Internet desde su inicio hasta la actualidad.



9. Realiza un diagrama conceptual de composición de la web. ¿Cuáles son los componentes de la www?



10. ¿Cuál es la arquitectura del sistema distribuido de documentos?

Es una arquitectura Cliente-Servidor distribuida de acoplamiento débil, que utiliza un esquema de arquitectura basada en recursos (*Resource-Oriented Architecture - ROA*).

11. ¿Cómo funciona el sistema www?

El sistema World Wide Web (WWW) funciona mediante una arquitectura cliente-servidor que utiliza el protocolo HTTP (o su versión segura HTTPS) para transmitir datos. El proceso básico incluye:

Solicitud: El usuario introduce una URL en un navegador web (cliente), el cual contacta con un servidor DNS para traducir el nombre de dominio a una dirección IP.

Comunicación: El navegador envía una solicitud al servidor web alojado en esa IP, solicitando los archivos de la página.

Respuesta y Renderizado: El servidor procesa la petición, envía los archivos (principalmente en lenguaje HTML) y el navegador interpreta el código para mostrar el contenido visual al usuario.

12. ¿Qué es HTML? ¿De donde proviene?

HTML (*HyperText Markup Language*): Es el lenguaje de marcado estándar utilizado para la estructuración y presentación del contenido de las páginas web. Define mediante elementos y etiquetas (*tags*) la semántica y jerarquía de un documento (títulos, párrafos, enlaces, tablas, formularios).

Este fue derivado e ideado por Tim Berners-Lee a partir de SGML (*Standard Generalized Markup Language*, ISO 8879), un estándar más amplio y complejo para la organización de documentos digitales.

13. ¿A que se le llama página web estática? ¿y página dinámica? ¿y página activa?

Página Estática: Su contenido está predefinido en archivos (HTML/CSS) y no cambia, a menos que un desarrollador modifique el código fuente manualmente.

Página Dinámica: El contenido se genera en tiempo real en el servidor (Server-side, ej. PHP, Python, Node.js) según la solicitud del usuario, su sesión o consultas a bases de datos.

Página Activa: Ejecuta código en el cliente (Client-side, ej. JavaScript) para modificar la interfaz y ofrecer interactividad directamente en el navegador del usuario sin necesidad de recargar la página.

14. ¿Qué es una URL? ¿Qué función cumple en la Web? ¿Qué es un servidor de archivos basado en URL?

URL (Uniform Resource Locator): Identificador sintáctico global que especifica tanto la ubicación de un recurso de red como el protocolo para acceder a él.

Función: Proporcionar un direccionamiento único y estandarizado para recuperar cualquier elemento dentro de la Web.

Servidor de archivos basado en URL: Es un servidor web configurado para mapear el path (ruta) de una URL entrante a la estructura jerárquica del sistema de archivos (file system)

local en el disco del servidor, o a una ruta virtual equivalente, sirviendo el archivo especificado directamente al cliente.

15. ¿Dónde y cómo se ejecuta el lenguaje HTML?

Dónde se ejecuta: Se interpreta e instruye exclusivamente en la máquina cliente, dentro del navegador web.

Cómo se ejecuta: El motor de renderizado (Rendering Engine como Blink, Gecko o WebKit) recibe el flujo de bytes de código HTML enviado por el servidor, los convierte a caracteres, luego a tokens, genera nodos y finalmente construye la estructura en memoria llamada DOM (Document Object Model), procesando CSS para pintar el resultado visual en la pantalla.

16. ¿Qué es un lenguaje script en la tecnología WWW?

Un lenguaje de script o de ordenes es un lenguaje de programación interpretado (o compilado JIT - Just-In-Time), típicamente de tipado dinámico, diseñado para automatizar tareas, manipular entornos o controlar aplicaciones web sin necesidad de compilación previa a código máquina binario.

Scripts Client-side: Ejecutados por el motor del navegador para dar interactividad a las páginas (ej. JavaScript).

Scripts Server-side: Ejecutados en el servidor web para procesar peticiones, conectarse a bases de datos y construir HTML dinámico (ej. PHP, Python, Ruby).

17. ¿Qué es un plugin? ¿Qué es una cookie?

Plugin: Es un módulo de software de terceros que se integra o acopla a un navegador web para extender sus funcionalidades originales y permitirle procesar contenidos que nativamente no soporta (ej. antiguamente Adobe Flash Player, Java Applets, lectores de PDF dedicados). Nota: Han sido prácticamente reemplazados por los estándares nativos de HTML5 y WebAssembly.

Cookie: Es una pequeña pieza de datos (par clave-valor junto a atributos de control) que un servidor web envía al navegador del cliente dentro de la cabecera HTTP Set-Cookie. El navegador almacena estos datos localmente y los reenvía de forma automática en cada subsiguiente solicitud HTTP hacia el mismo dominio (Cookie: ...), permitiendo mantener un estado (stateful) en un protocolo inherentemente sin estado como HTTP (utilizadas para sesiones, carritos de compras y rastreo).

18. ¿Qué es JavaScript? ¿Dónde y quién ejecuta JavaScript? ¿Cómo nació JavaScript?

¿Qué es JavaScript? Es un lenguaje de programación interpretado, orientado a objetos basado en prototipos, dinámico y multiparadigma. Es el lenguaje estándar de programación nativo de la Web.

Dónde y quién lo ejecuta: Típicamente lo ejecuta el cliente mediante el motor de JavaScript integrado en el navegador web (ej. V8 en Chrome/Edge, SpiderMonkey en Firefox, JavaScriptCore en Safari). En el servidor, es ejecutado por entornos de ejecución como Node.js o Deno.

Historia: Fue diseñado y creado por Brendan Eich en solo 10 días en mayo de 1995 mientras trabajaba en Netscape Communications. Inicialmente se llamó Mona, luego LiveScript y finalmente se renombró a JavaScript por razones comerciales de marketing relacionadas al auge de Java. Fue estandarizado posteriormente bajo la especificación ECMAScript.

¿Qué es DHTML? ¿ y XHTML? 25. ¿Qué es XML ? Explique detalladamente.

DHTML (Dynamic HTML): Término utilizado a finales de la década de 1990 para describir la combinación de HTML, CSS, JavaScript y el DOM para crear sitios web interactivos y dinámicos sin necesidad de recargar la página completa.

XHTML (Extensible HyperText Markup Language): Reformulación del lenguaje HTML4 bajo las estrictas reglas sintácticas del estándar XML. Exigía que todas las etiquetas estuvieran cerradas, en minúsculas y adecuadamente anidadas. Perdió popularidad con la llegada de la flexibilidad semántica de HTML5.

XML (*Extensible Markup Language*): Es un metalenguaje de marcado extensible desarrollado por el W3C para estructurar, almacenar e intercambiar datos de forma legible tanto para personas como para máquinas.

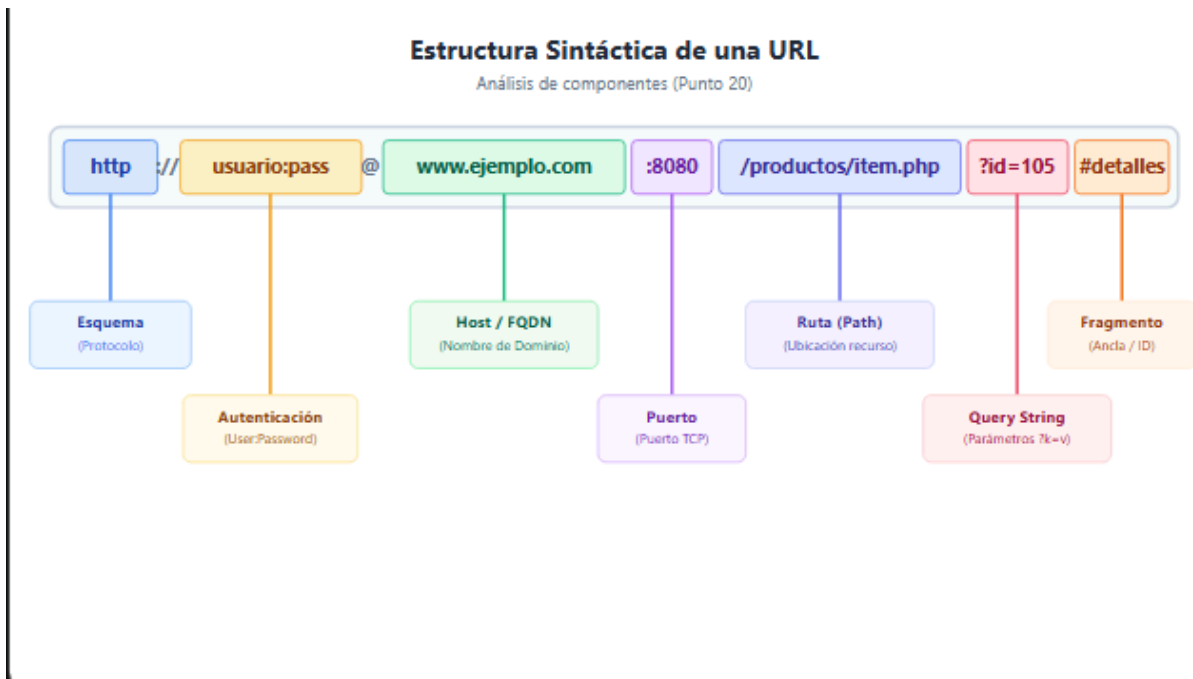
19. ¿Qué son los Servicios Web? De un ejemplo de aplicación utilizando Servicios WEB.

Un Servicio Web (Web Service) es un conjunto de protocolos y estándares que permite la interacción y el intercambio de datos de máquina a máquina (M2M) sobre una red, utilizando protocolos web como HTTP.

Ejemplo práctico de aplicación: Un sistema de comercio electrónico (e-commerce) desarrollado en Node.js que, al momento del cobro, realiza un consumo mediante peticiones HTTP/REST (enviando payloads JSON) a la API de una pasarela de pagos externalizada (ej. Mercado Pago o Stripe) para autorizar la transacción con tarjeta de crédito de manera remota.

20. ¿Qué es una URL? ¿Qué función cumple en la web? ¿Cuál es la estructura de una URL?

Concepto y Función: La URL (Uniform Resource Locator) es un tipo de URI que identifica de manera única la ubicación de un recurso accesible en la red y el protocolo mediante el cual debe solicitarse.



21. ¿Qué es un servidor de archivos basado en URL?

Un servidor de archivos basado en URL es la abstracción en software donde el servidor traduce de manera directa los elementos del path de la URL enviada en la solicitud HTTP a rutas de directorios locales dentro del sistema de almacenamiento del servidor.

22. ¿Cuál es el formato de una URL en IPv6?

Debido a que las direcciones IPv6 contienen caracteres de dos puntos (:) para separar sus bloques hexadecimales, la sintaxis estándar de la URL exige que la dirección IPv6 sea encerrada obligatoriamente entre corchetes angulares [] para evitar ambigüedades con el delimitador del número de puerto.

23. ¿Qué es un Dominio www? ¿Qué relación tiene con una URL?

Dominio WWW: Es una cadena de texto alfanumérica que representa un nombre amigable o mnemotécnico asignado a uno o más servidores con direcciones IP dentro del sistema de nombres de dominio (DNS) (ejemplo: uader.edu.ar).

Relación con la URL: El nombre de dominio conforma el componente de Host (hostname) dentro de la estructura de una URL. La URL utiliza el dominio amigable para evitar obligar al usuario a memorizar las direcciones IP numéricas subyacentes.

24. ¿Cuál es la función del DNS en la WEB?

El DNS (Domain Name System) es una base de datos distribuida y jerárquica cuyo rol fundamental es realizar la resolución de nombres de dominio a direcciones IP (y viceversa).

25. ¿Cuál es la relación entre HTML, HTTP y HTTPS?

Es una relación de Contenido, Transporte y Seguridad:

HTML: Es el contenido/payload (el formato de datos que contiene el mensaje).

HTTP: Es el protocolo de transporte de capa de aplicación que define las reglas para empaquetar, transferir y recibir esos documentos HTML entre cliente y servidor.

HTTPS (HTTP Secure): Es el mismo protocolo HTTP pero ejecutándose de manera cifrada sobre una capa de seguridad criptográfica (TLS/SSL), garantizando autenticidad, confidencialidad e integridad en el tránsito de los datos.

26. ¿Cuáles son las versiones de HTTPS y cuáles sus diferencias?

Técnicamente, HTTPS no posee versiones propias por sí mismo, sino que sus versiones y evolución dependen del protocolo de cifrado subyacente de la capa de transporte sobre el cual opera: SSL (Secure Sockets Layer) y su sucesor TLS (Transport Layer Security).

A continuación, se detallan las distintas versiones de los protocolos de cifrado que dan vida a HTTPS y sus diferencias clave:

SSL 2.0 y SSL 3.0 (Obsoletos e Inseguros):

Estado: Totalmente prohibidos y fuera de uso en la web moderna.

Diferencias: Fueron los primeros protocolos desarrollados por Netscape en la década de 1990. Presentaban graves fallas de diseño y vulnerabilidades críticas de seguridad (como los ataques POODLE y BEAST), por lo que quedaron completamente obsoletos.

TLS 1.0 y TLS 1.1 (Obsoletos):

Estado: Deprecados oficialmente por el IETF y desactivados en los navegadores actuales.

Diferencias: TLS 1.0 surgió en 1999 como la evolución directa de SSL 3.0. Aunque mejoraron la seguridad básica, dependían de algoritmos de cifrado débiles (como MD5 y SHA-1) vulnerables a ataques de degradación (downgrade attacks).

TLS 1.2 (Ampliamente Utilizado):

Estado: Estándar maduro, seguro y aún ampliamente compatible.

Diferencias: Publicado en 2008, introdujo mejoras significativas en criptografía: soporte para algoritmos Hash más seguros (SHA-256), cifrados autenticados (AEAD) y mayor flexibilidad para negociar claves avanzadas. Su proceso de negociación inicial (handshake) requiere 2 RTT (Round Trip Times) para establecer el canal seguro.

TLS 1.3 (Estándar Actual y Recomendado):

Estado: Estándar moderno de la Web.

Diferencias:

Mayor Velocidad: Reduce el tiempo de handshake a solo 1 RTT (e incluso 0 RTT para reconexiones), acelerando notablemente el establecimiento de la conexión.

Mayor Seguridad: Eliminó de forma definitiva algoritmos criptográficos obsoletos o vulnerables (como RC4, interrupciones de claves RSA tradicionales y SHA-1) y exige Perfect Forward Secrecy (PFS).

Mayor Privacidad: Cifra la negociación del certificado digital desde fases mucho más tempranas del proceso, impidiendo que terceros inspeccionen datos de la conexión.

27. ¿Qué es MIME?

MIME (Multipurpose Internet Mail Extensions): Es un estándar original de correo electrónico extendido a HTTP para indicar la naturaleza y formato exacto de un recurso o archivo transmitido sobre la red.

28. Explique cómo es una solicitud HTTP.

Una solicitud (o petición) HTTP es un bloque de texto plano formateado bajo el estándar del protocolo que el cliente envía al servidor a través del socket TCP.

Estructura de una solicitud HTTP:

Línea de Solicitud (Request Line): Contiene el Método HTTP, el Path del Recurso solicitado y la Versión del Protocolo.

Cabeceras de Solicitud (Request Headers): Pares clave-valor que envían metadatos sobre el cliente, soporte de compresión, cookies, tokens de autenticación, etc.

Línea en blanco: Un carácter de retorno de carro y salto de línea (`\r\n`) que separa las cabeceras del cuerpo.

Cuerpo de la Solicitud (Message Body / Payload): Opcional (común en peticiones POST o PUT), contiene los datos enviados al servidor.

29. ¿Cuáles son los métodos de HTTP? Enumere y describa el funcionamiento de cada uno.

Los métodos HTTP indican la acción semántica que el cliente solicita realizar sobre un recurso identificado en el servidor.

GET:

Descripción: Solicita la representación de un recurso específico. Únicamente debe recuperar datos y no debe tener efectos secundarios en el servidor (operación segura y idempotente).

POST:

Descripción: Envía datos al servidor para que procese y cree un nuevo recurso subordinado o ejecute una acción. No es idempotente (reiterar la petición crea múltiples registros).

PUT:

Descripción: Reemplaza completamente el recurso destino con la carga útil (payload) suministrada en la petición. Es idempotente.

DELETE:

Descripción: Elimina el recurso especificado por la URL en el servidor. Es idempotente.

PATCH:

Descripción: Aplica modificaciones parciales a un recurso existente (a diferencia de PUT que lo reemplaza por completo).

HEAD:

Descripción: Solicita una respuesta idéntica a la de una petición GET, pero sin el cuerpo de la respuesta (message body). Útil para verificar metadatos, existencia o caché de un recurso sin descargar su contenido.

OPTIONS:

Descripción: Describe las opciones de comunicación y métodos permitidos para el recurso objetivo (muy utilizado en mecanismos de seguridad CORS pre-flight).

TRACE:

Descripción: Realiza una prueba de bucle de retorno (loopback) a lo largo de la ruta hasta el recurso, permitiendo ver qué reciben los servidores intermedios (proxies).

CONNECT:

Descripción: Establece un túnel de conexión de red bidireccional con el servidor indicado, utilizado comúnmente para establecer comunicaciones cifradas HTTPS a través de un servidor proxy no cifrado.